AgentPProf: Operation-Stack Profiling for AI Agent Traces

Abstract

AI-agent executions are no longer well described by one prompt, one session, or one trace tree. A single run can contain prompts, tool calls, GUI actions, browser actions, process events, plans, subagents, safety labels, failure labels, and human subtask boundaries. Existing observability systems usually make one boundary primary, such as a session, span, or prompt tree. Classic profilers make another boundary primary, such as a function call stack. This paper argues that agent profiling needs a smaller abstraction: an `operation` is a weighted record with fields, and an `operation stack` is a recursive projection over those fields chosen at query time. Aggregation is not a separate object or a fixed trace tree; it is the result of choosing a stack shape over the same operations. This is not merely a claim about query-time aggregation, which classic profilers and trace processors also support in different forms. The scoped novelty is the combination of an agent-operation record model, recursive multi-field operation-stack projections, and a hidden-label cross-dataset localization benchmark.

We implement this model in the Rust AgentPProf profiler and evaluate it through four paper-facing experiments rather than a chronological run list. E1 tests coverage and recursive folding: across 15 lightly sampled public labeled sources, AgentPProf maps 47,590 samples into the same operation layer, and the same 13,265 operations fold from 9 dataset stacks to 57 phase, 226 semantic, 455 action, or 3,757 fixed-session stacks without changing the input operations. E2 is the main hidden-label localization benchmark. On AgentRewardBench, SATraj-OS, AgentNet, and OSWorld-Human, we build six tasks over 34,539 operations and 3,699 positives, score 144 view/ranker policies, and find that task-query operation-stack profiling inspects median 0.0937 top-five work versus 1.0 for flat summaries, improves top-five recall over fixed-session drilldown on 5 of 6 tasks, and reduces median group fragmentation from 285.0 to 157.5 groups. E3 isolates mechanisms and profile-configuration actionability. Rank-feature, stack-depth, mapping, transfer, diagnostic-lens, case-packet, and counterfactual analyses show that the useful configuration varies by task and objective: operation-stack views are a Pareto point, not a universal default. Executable profile-spec patches improve AP, top-five lift, and first-positive work on 5 of 6 tasks; a supervised held-out boundary-field ablation improves OSWorld-Human AP from 0.2402 to 0.2583 and reduces groups from 108 to 74, while preserving work counterpoints. E4 checks replayability and offline cost: deterministic replay covers 76 tracked profile specs, while source-status, number, rubric, guardrail, and two-abstraction checks remain artifact hygiene rather than empirical profiler evidence.

The supported claim is therefore a scoped profiling claim: operation-stack profiling can localize, rank, and explain task-relevant failures, quality problems, and semantic boundaries in public labeled traces, with less inspection work than flat summaries and less fragmentation than fixed-session drilldown/span-tree proxy views. The paper does not claim human-productivity gains, automatic discovery of all intent boundaries, metric dominance, or complete compatibility with existing trace ecosystems.

1 Introduction

AI-agent traces mix many objects that traditional profiling and tracing systems keep separate. A coding agent may ask a model for a plan, call a shell tool, launch a subagent, inspect a file, and retry a failing command. A web or desktop agent may click, type, observe a page, repeat an action, and receive a benchmark label saying that the action was unsafe, redundant, incorrect, or the start of a human subtask. If the profiler fixes the hierarchy at prompt, session, or span boundaries, these questions become hard to ask: which phases concentrate unsafe operations, which action families cause redundant steps, and which groups of operations cross sessions but share the same failure mode?

AgentPProf takes the opposite position. The profiler stores a sequence as operations, where prompts, sessions, tool calls, processes, syscalls, GUI actions, and plan steps are all operation shapes or operation fields. Users then choose an operation stack to fold the same operations at different depths. One query may group by `task,phase,action`; another may group by `environment,safety`. Mapping and tagging are first-class field-derivation steps, but they do not add a third profiler object. They only write fields before the stack query runs. Thus the novelty is the abstraction that connects stack shape to aggregation: changing the stack changes what is comparable, localizable, and actionable without changing the underlying trace object model.

This paper makes a profiling claim, not a human-productivity claim. A top-tier profiling paper must show that the profiler attributes, localizes, and ranks problems against explicit workload labels, and that the output leads to optimization insight with better label-scored ranking or inspection cost than baselines. For agents, the claim is:

> *Operation/operation-stack profiling provides label-scored localization and ranking for task-relevant failures, quality problems, and semantic boundaries in real labeled agent traces, while requiring less inspection work than flat summaries and less fragmentation than the fixed-session drilldown used here as a span-tree proxy.*

We evaluate this claim with public labeled traces rather than a human study. A human or agent analyst study would be useful for productivity claims, but it is not required to test label-scored profiler fidelity. Our evaluation therefore uses hidden labels as the oracle for ranked profile groups and is organized around four core experiments. E1 tests operation coverage, recursive stack folding, field derivation, and human-boundary alignment under the same two-object model. E2 is the main hidden-label localization and ranking benchmark. E3 isolates mechanisms and actionability through ranker, stack, mapping, transfer, case, and profile-spec patch ablations. E4 checks reproducibility and offline cost, with claim-integrity guardrails kept as artifact hygiene. Individual R-runs are provenance for these experiments, not the paper-facing structure.

2 Design

2.1 Operations

An `operation` is the only sample-level object in AgentPProf. It is a weighted record with string fields. The same representation can encode a prompt, a tool call, an API call, a browser action, a PyAutoGUI action, a process event, or a derived label such as `looping=yes` or `step_correct=incorrect`. External datasets enter the system as operation JSONL. Local Codex or Claude sessions can also be exported as an agent-session trace, imported back, or converted to operation JSONL before profiling.

2.2 Operation Stacks

An `operation stack` is an ordered list of fields. Folding operations by that list yields the usual folded-stack representation, but the stack is chosen at query time. The same operation file can be folded as a flat task summary, a fixed-session tree, a dataset-native hierarchy, a raw action stack, a semantic operation stack, or an oracle label-drilldown. Fixed sessions and spans are therefore baselines and exchange containers, not core profiler abstractions.

2.3 Mapping and Tagging

Mapping and tagging derive new operation fields before folding. For example, desktop actions such as `type` and `key` can map to `phase=input`, while tool/API rows can map to a tool phase before generic action rules run. The Rust CLI exposes inline rules, rule files, profile specs, operation predicates, and stack overrides. `–where` predicates run after mapping and before stack construction, implementing the query predicate without adding a profiler object. Profile specs record operation files, mappings, predicates, views, stacks, and outputs for reproducibility, while command-line flags can still override the stack query.

3 Evaluation Setup

Table 1 summarizes the public trace families. We do not create new test sets, sync complete image corpora, or depend on gated data. The 15 sources establish generality, while the main hidden-label ranking results use the four oracle-rich families with failure, safety, quality, and human-boundary labels.

For R320, we build six tasks: AgentRewardBench looping, AgentRewardBench side-effect, SATraj unsafe operation, AgentNet incorrect step, AgentNet redundant step, and OSWorld-Human group start. Each task has an oracle-positive field, but non-oracle rankers never read that field. We compare six views: flat, fixed-session, dataset-native hierarchy, raw action stack, semantic operation stack, and label-drilldown. We compare four rankers: width, visible-risk, query-aware, and oracle upper bound. Label-drilldown and oracle upper bound are marked as hidden upper bounds.

The metrics match a profiler paper's main claim. Fidelity uses precision@k, recall@operation-budget, F1@k, average precision as a block-level AUPRC-style score, nDCG over positive operation counts, and work-to-first-positive. Fragmentation uses group count, positive group count, and work or groups to reach 50% positive recall. Actionability uses per-task comparisons among stack fields, mapping choices, ranker choices, and oracle headroom. Reproducibility uses repeated Rust `agentpprof –profile-spec` executions over tracked specs and tracked operation JSONL inputs, checking semantic output hashes, sample counts, unique-stack counts, runtime, and output size. Build time is excluded from the profiler runtime measurement.

Table 2 gives the paper-facing evaluation structure. R-numbered runs are provenance, ablations, counterpoints, or audit gates rather than the paper's main organization. R360 regenerates this four-row result table from tracked artifacts and emits JSON, CSV, Markdown, HTML, and a LaTeX table fragment. It is a table-provenance gate, not a new empirical result or profiler rerun.

4 Results

4.1 E1: Coverage and Recursive Folding

Evidence contract: claim, the operation/operation stack model covers heterogeneous traces without prompt/session/tool-bound units; oracle, public labels and native fields, especially OSWorld-Human boundaries and AgentNet quality; baseline, dataset-native/no-map/fixed-session stacks plus replay and trace round trips; metric, coverage, stack range, compression, boundary F1/V-measure, and equality; counterpoint, configurable operation stack folding, not complete compatibility or latent-intent recovery. Prompt, session, tool, process, and syscall records are operation fields, not new profiler objects. RQ1 asks whether one operation layer can be folded recursively into task, phase, action, boundary, and fixed-session-shaped stacks by changing mappings and stack fields, without changing the input records or adding a third profiler object.

The 14 core public sources produce 42,590 operations. Adding the supplemental ScaleCUA stream brings the smoke set to 47,590 operations. These operations cover web, mobile, desktop, software, API, tool-agent-user dialogue, expert failure labels, safety labels, human grouped-action labels, and desktop step-quality labels. No source required a prompt-specific, GUI-specific, or safety-specific profiler object.

Recursive stack-depth sweep. On the same 13,265 operations, a depth sweep folds to 9 dataset stacks, 57 phase stacks, 226 tool/semantic stacks, 455 action stacks, and 3,757 fixed-session stacks. A fixed session boundary therefore increases stack count by 417.4× relative to dataset depth. On AgentNet, a tracked profile spec folds 16,741 operations into 608 diagnostic stacks, while a CLI stack override changes the output to 83 stacks without changing the operation input. The result supports query-time recursive stack choice rather than a fixed prompt/session hierarchy. R321 adds an implementation probe for query predicates: profile specs over the tracked R300 operation file select 729/729, 714/714, and 4,285/4,285 expected operations with mapping-derived `where_rules` before folding. R322 adds the corresponding Rust JSON ranking surface: profile specs with visible `rank_rules` emit ranked operation-stack groups over the same six tasks, improving AP over width on 4 of 6 tasks while preserving counterexamples for richer group-level rankers. R323 adds a `rank_mode` probe: score-first ranking improves AP over width-boost on 4 of 6 tasks and reduces SATraj first-positive work from 0.6376 to 0.0842, while side-effect and OSWorld-Human remain counterexamples. R324 adds operation-level rank features: `rank_op_rules` match visible mapped `field=value` operation tokens and aggregate matched weight inside each folded group. On

Table 1: Public labeled trace families used by the evaluation. The main R320 hidden-label profiler benchmark uses the four oracle-rich rows in the bottom half.

Family	Native labels	Access path	Evaluation role
WebLINX, Mind2Web, AgentTrek, WebShop	Web actions, demos, tasks, rewards, action history	Hugging Face viewer or public shards	Web navigation coverage
API-Bank, ToolBench, tau-bench	API calls, tool messages, solution paths, outcomes	Hugging Face rows or repo JSONL	Tool/API and dialogue coverage
SWE-agent	Issue trajectory, command actions, target outcome	Hugging Face viewer	Software-agent coverage
AndroidControl, GUI-Odyssey, ScaleCUA	Mobile or GUI actions, instructions, history context	Public/HF samples	Mobile and GUI coverage
AgentRewardBench	Expert success, side-effect, looping, optimality	HF annotations plus cleaned BrowserGym steps	Failure and looping tasks
SATraj-OS	Desktop actions, safety, reward, attack type	HF safety config	Safety task
OSWorld-Human	Human single-action and grouped-action traces	Public GitHub JSON	Human-boundary task
AgentNet	Human desktop PyAutoGUI, step correctness, redundancy	HF repo JSONL streaming	Step-quality tasks

Table 2: Paper-facing core experiments and generated headline evidence. R360 regenerates the table from tracked artifacts; individual R-runs remain provenance and counterpoint records.

Core experiment	Workload	Generated headline evidence	Scoped conclusion
E1: coverage and recursive folding	15 public labeled trace families / 47,590 operations, plus local-session and standard-trace exchange fixtures	R286 folds the same 13,265 operations across 8 depths, from 9 to 3,757 stacks; R285 has 6/9 positive leave-dataset-out stack reductions; R342 has 12/12 prompt/session-free profile specs; R353 round-trips 512 real operations with 512 samples and 11 stacks preserved.	Two abstractions cover heterogeneous traces; trace containers and mappings are not profiler objects.
E2: hidden-label localization and ranking	Six oracle-backed tasks over AgentReward-Bench, SATraj-OS, AgentNet, and OSWorld-Human	R320 covers 6 tasks, 4 datasets, 34,539 operations, 3,699 positives, and 144 policies. Operation-stack query-aware uses 0.0937 top-five work versus flat 1.0, beats fixed-session top-five recall on 5/6 tasks, and uses 157.5 median groups versus fixed-session 285.	Supported as a hidden-label profiler benchmark with baseline tradeoffs, not human utility.
E3: mechanism and actionability	The same six tasks plus held-out OSWorld-Human boundary-backend operations	R354 accepts 5/6 executable profile-spec patches, with median AP delta 0.0376 and top-five lift delta 0.5750. R358 improves OSWorld-Human boundary AP to 0.2583 versus 0.2402 and reduces groups to 74 versus 108, while preserving top-five-work and first-positive-work counterpoints.	The profiler exposes stack, field, ranker, and profile-spec knobs; it is not an automatic selector or boundary detector.
E4: reproducibility and offline cost	76 tracked profile specs over tracked operation JSONL inputs	R328 gives 76/76 semantic and 76/76 raw-byte deterministic specs over 152 invocations, with median runtime 1.601s and p95 2.767s.	The offline profiling artifact is replayable and cheap enough for artifact evaluation; this does not measure live capture overhead, human productivity, or trace-ecosystem compatibility.

semantic stacks, AP improves over width on 5 of 6 tasks and first-positive work improves on 5 of 6 tasks; on coarser stacks, AP improves on 4 of 6 tasks while group counts fall. R325 replays the same scrubbed profiler input and stack projections with newly generated profile specs that remove one visible feature at a time. It finds 7 critical feature instances, 3 misleading feature instances, and a stack-depth tradeoff: coarse stacks are AP-preferred on 2 of 6 tasks but reduce group counts on all six tasks. R326 tests whether these rank policies are brittle hand-tuned weights. A global equal-weight visible feature bank improves AP over width on 4 of 6 semantic tasks and 5 of 6 coarse tasks. Equal-weight task policies stay within 0.02 AP of their weighted counterparts on 8 of 12 task/depth variants. R325-guided repairs improve AP on 2 of 3 misleading-feature cases and first-positive work on 2 of 3 cases; 1 of 3 improves both metrics. This is post-hoc actionability evidence, not a label-free deployment ranker. R329 adds a target-held-out transfer probe. Candidate policies include the global equal feature bank and six source-task equal-weight policies. Leave-task and leave-dataset selection use only non-target task scores, while target labels are reserved for final scoring. Both protocols improve semantic/coarse AP over width on 4 of 6 tasks and semantic first-positive work on 6 of 6 tasks, but side-effect, SATraj, and boundary counterexamples remain; the result isolates policy transfer from target-label tuning without claiming a universal ranker.

Field derivation and boundary probes. Held-out mapping rules improve compression from 14.049 to 19.091 on 3,990 operations. Leave-dataset-out mapping reduces unique stacks on 6 of 9 held-out datasets with no negative stack-reduction regressions after API/tool phase precedence is fixed. A supervised OSWorld-Human boundary backend reaches F1 0.7735 on held-out adjacent pairs and writes predicted boundary fields that the Rust profiler folds as ordinary operation fields. Boundary backends therefore extend field derivation, not the profiler object model.

Human-boundary oracle. OSWorld-Human gives the strongest direct boundary oracle. On 320 exact-aligned tasks with 4,011 operations and 2,075 human groups, grouped-depth stacks reduce 482 action stacks to 106 grouped stacks. A conservative `group_pattern` projection has human-group boundary F1 0.627 with precision 1.0 and recall 0.456. This is not complete intent discovery, but it shows

that the same single-action sequence can be folded at action depth or human-group depth and scored against a human boundary oracle.

4.2 E2: Hidden-Label Localization and Ranking

Evidence contract: claim, operation-stack profiling localizes and ranks real labeled failures, quality problems, safety issues, and semantic boundaries; oracle, 3,699 positives over 34,539 operations; baseline, flat, fixed-session drilldown/span-tree proxy, dataset-native, raw-action, operation-stack, label-drilldown, and oracle upper bound; metric, precision@k, recall, F1, AUPRC-style AP, nDCG, budget recall/F1, work-to-first-positive, and groups; counterpoint, flat and dataset-native views can win broad-recall or nDCG goals, so the claim is Pareto tradeoff rather than metric dominance. RQ2 asks whether hot stacks and top-ranked groups correspond to hidden positives while requiring less inspection work than flat summaries and less fragmentation than fixed-session drilldown, used here as a span-tree proxy.

Table 3 gives the main R320 accuracy result. Flat summaries recover all positives, but only by inspecting the whole task, so their median work@5 is 1.0 and their work-to-first-positive is 1.0. Fixed-session drilldown finds an early positive with median work-to-first-positive 0.0044 and median work@5 0.0163, but its median recall@5 is only 0.0233 and it fragments the workload into 285.0 groups. Query-aware operation stacks inspect median 0.0937 work in the top five groups, reach median recall@budget30 of 0.3900, and reduce the median group count to 157.5.

The strongest comparison is not universal dominance. Operation stacks improve top-five recall over fixed-session query-aware drilldown on 5 of 6 tasks and reduce group fragmentation from median 285.0 to 157.5 groups, but fixed-session often finds the first positive with less work. Operation stacks also save work against flat summaries on all tasks, but flat remains a complete full-recall counterpoint. Dataset-native hierarchy sometimes has higher recall, but at high inspection work. These mixed results are the desired profiling-paper evidence: operation stacks occupy a useful Pareto region between flat summaries and fixed-session drilldown.

R330 audits uncertainty over these R320 comparisons without rerunning the profiler or creating data. It bootstraps paired task-family deltas for 10,000 resamples. Against flat width, operation-stack query-aware has stable positive AP delta (mean 0.1219, 95% CI [0.0406, 0.2175]), stable lower top-five work (mean -0.8168, CI [-0.9653, -0.6568]), stable higher 30% budget recall (mean 0.4510, CI [0.3450, 0.6143]), and stable lower work-to-first-positive (mean -0.9303, CI [-0.9855, -0.8650]). Against fixed-session query-aware, operation stacks have stable higher top-five recall (mean 0.1801, CI [0.0542, 0.3127]), higher top-five F1 (mean 0.1702, CI [0.0549, 0.2870]), and fewer groups (mean -178.0, CI [-340.0, -39.0]). Against width-only operation stacks, query-aware ranking has stable higher AP (mean 0.0932, CI [0.0110, 0.2184]) and lower top-five work (mean -0.3101, CI [-0.4209, -0.1997]). The same audit marks flat top-five recall, fixed-session first-positive work, and raw-action mapping comparisons as mixed or counterpoints; it is a task-level uncertainty audit, not a per-operation independence or human-utility claim.

R331 adds a prevalence and group-size negative control. It keeps each visible policy's groups and ranking order fixed, then randomly reallocates each task's hidden positive operations across same-size group positions for 2,000 permutations. Operation-stack query-aware AP exceeds the 95% null on 6 of 6 tasks, with median observed-minus-null AP 0.0759; 30% budget recall exceeds the null on 5 of 6 tasks, with median delta 0.0904. These results show that the main AP and budgeted-recall signals are not explained by prevalence and group sizes alone. The same audit is also a boundary: top-five precision exceeds the null on only 3 of 6 tasks, work-to-first-positive on 0 of 6, width-only operation-stack AP on 5 of 6, and fixed-session/raw-action AP on 6 of 6. R331 therefore supports mechanism isolation and baseline realism, not single-view dominance or a label-free deployment ranker.

R332 asks whether a single visible view/depth policy can be fixed after R320. It reuses the R320 visible policy scores, does not rerun profiling, and excludes oracle policies from selection. Best visible AP is split across operation-stack, fixed-session, and dataset-native views: 3 of 6, 2 of 6, and 1 of 6 tasks, respectively. Best top-five F1 is more dispersed: raw-action stacks win 2 of 6 tasks, while dataset-native, fixed-session, flat, and operation-stack each win 1 of 6. Operation-stack query-aware is best for AP on 3 of 6 tasks and best for 30% budget recall on 3 of 6 tasks, but only 1 of 6 for top-five F1 and 2 of 6 for work-to-first-positive. It still reduces group fragmentation relative to fixed-session query-aware on 4 of 6 tasks, with median group ratio 0.5543. Leave-task source selection does not solve view choice universally: selected-equals-best is 0 of 6 for AP and 0 of 6 for top-five F1. R332 therefore supports the design implication that view, stack fields, predicates, and rankers must remain query-time configuration over the same operations. Fixed-session drilldown is the current span-tree proxy; real span-tree imports remain future baselines, and dataset-native hierarchies remain baseline or drilldown views, not profiler abstractions.

R333 turns the same comparison into an inspection-efficiency curve. It reruns the R320 local scorer over the tracked operation JSONL inputs, yielding 810 visible top-k/work-budget inspection points and 252 task-policy curve rows; it does not fetch, sync, or create data. Within a 30% inspected-work budget, operation-stack query-aware reaches median recall 0.3900, compared with 0.0000 for flat width, 0.3559 for fixed-session query-aware, 0.3377 for dataset-native query-aware, and 0.3325 for raw-action query-aware. At 20% work, the same operation-stack policy reaches median recall 0.2763 versus 0.2422 for fixed-session query-aware. Against flat width, operation-stack query-aware uses lower top-five inspected work on all six tasks and has positive 30%-budget recall where flat has none. Against fixed-session query-aware, it has higher top-five recall on 5 of 6 tasks and fewer groups on 4 of 6 tasks, but lower work-to-first-positive on only 2 of 6 tasks. R333 therefore supports an inspection-cost and fragmentation tradeoff, not dominance on every cost metric.

R334 separates the fragmentation part of that claim from operation-work cost. It reads the tracked R320 policy scores and R333 inspection curves; it does not fetch, sync, create, or relabel data. Against fixed-session query-aware, operation-stack query-aware reduces total groups and positive groups on 4 of 6 tasks, and reaches 50% positive recall with fewer ranked groups on 5 of 6 tasks (median

Table 3: R320 hidden-label profiler benchmark. Hidden policies read oracle labels and are included only as upper bounds. WTFP is work-to-first-positive.

Policy	Hidden	AP	nDCG	P@5	R@5	F1@5	Work@5	R@30%	WTFP
flat:width	0	0.1678	1.0000	0.1678	1.0000	0.2868	1.0000	0.0000	1.0000
fixed-session:query-aware	0	0.3476	0.7642	0.2555	0.0233	0.0427	0.0163	0.3559	0.0044
dataset-native:query-aware	0	0.3572	0.7445	0.1712	0.8665	0.2822	0.8539	0.3377	0.0582
raw-action:query-aware	0	0.2744	0.5012	0.2513	0.2442	0.2195	0.1507	0.3325	0.0374
operation-stack:width	0	0.1610	0.9364	0.1398	0.4746	0.2147	0.5424	0.3372	0.1694
operation-stack:query-aware	0	0.3117	0.5487	0.1991	0.1880	0.1981	0.0937	0.3900	0.0378
operation-stack:oracle-upper	1	0.5993	0.6372	0.8984	0.3004	0.4029	0.0441	0.5640	0.0122
label-drilldown:oracle-upper	1	1.0000	1.0000	1.0000	0.6703	0.7972	0.1150	1.0000	0.0377

delta -31.5 groups). At the same 30% inspected-work budget, it inspects fewer groups on 5 of 6 tasks (median delta -54.0 groups) while its median recall delta is 0.0275. The same audit preserves the fixed-session work counterpoint: operation stacks have lower work-to-50%-recall on only 1 of 6 tasks, lower top-five work on only 2 of 6 tasks, and lower work-to-first-positive on only 2 of 6 tasks. Thus the supported claim is fixed-session fragmentation reduction and budgeted-recall tradeoff, not work dominance over span-tree drilldown.

4.3 E3: Mechanism and Actionability

Evidence contract: claim, the profiler exposes actionable knobs in stack fields, mapping/tagging rules, rankers, profile specs, and boundary-derived operation fields; oracle, hidden labels used only after profiling, plus held-out OSWorld-Human boundary positives for R358; baseline, default/patched specs, visible rankers, transfer policies, learned-boundary, fixed-session, and semantic-width stacks; metric, accepted patches, AP delta, top-five lift, first-positive-work, group reduction, and top-five work counterpoint; counterpoint, OSWorld-Human needs boundary fields, so this is not automatic boundary discovery, not automatic patch selection, and not a universal selector. RQ3 asks which mechanisms explain the localization gains, and whether profile-guided changes to stack fields, rank features, mappings, or boundary fields improve the output without leaking hidden labels into profiler inputs.

R320 turns localization scores into an optimization diagnosis rather than only a leaderboard, summarized in Table 4. SATraj unsafe is the cleanest operation-stack win: environment, phase, and action fields plus query-aware risk ranking give the best visible top-five F1. Looping exposes the value and limit of `repeat_signal`: it helps ranking, but positives are prevalent, so prevalence-aware ranking remains the optimization. Side-effect has large oracle headroom and points to deeper write/input mappings. AgentNet step-quality tasks need action-family localization followed by fixed-session examples. OSWorld-Human needs boundary-derived fields rather than action depth alone. Thus the first actionability result is not that one view wins; it is that the profiler identifies which stack field, mapping, ranker, or drilldown should change for each task family.

The Rust mechanism ablations isolate where those diagnoses come from. Stack-text rules and rank-mode variants (R322/R323) are replayable from profile specs, but they are too coarse for safety and side-effect tasks. Operation-level rank features (R324) close that gap by matching visible mapped operation tokens before folding: semantic-stack feature ranking improves AP over width on 5 of 6 tasks, top-five lift on 4 of 6, and first-positive work on 5 of 6; coarse depth improves AP on 4 of 6 while reducing groups. Leave-one-feature and robustness ablations (R325/R326) make the knobs concrete: `repeat_signal`, write-action features, and `status=success` are critical in different families, while misleading safety-like or input-phase features can hurt AP or first-positive work. Equal/global feature banks often preserve the gains, and R325-guided repairs improve AP on 2 of 3 misleading-feature cases, but these repairs use offline scored feedback and are not an automatic learned ranker.

The view and policy audits show the same mechanism at the analysis-surface level. Best AP and best top-five F1 are split across operation-stack, fixed-session, dataset-native, raw-action, and flat views; leave-task source selection cannot reliably pick the best visible view. Nevertheless, the operation-stack query-aware policy remains Pareto-frontier on all six tasks, beats flat top-five work on all six, beats fixed-session top-five recall on 5 of 6, and reaches fewer groups-to-50% positives than fixed-session on 5 of 6. At a 25% recall target it covers all six tasks with median work 0.2000 versus flat 1.0000 and median 16.0 groups versus fixed-session 50.0; at 50% recall, other depths or views become the best-work counterpoints. Sequence and oracle-depth audits add the inspection unit below and above individual groups. R355 shows that at a 30% operation budget, operation-stack query-aware reaches median 0.4669 positive-session recall versus fixed-session 0.3230, and across 24 oracle-depth rows it has median 0.4342 budget-30 positive-unit recall, beats fixed-session unit recall on 20/24 rows, and reaches 50% positive units with fewer groups on 22/24. Positive-run proxy units are derived episode units, not human intent boundaries. R355's fixed-session depth-gap counterpoint remains, so this is depth-aware triage, not complete intent-boundary recovery. R344 also keeps the metric surface honest: it records 30 supporting baseline-metric comparisons, 16 explicit counterpoints, and nDCG cases where flat or dataset-native views can win.

The actionability portfolio turns those comparisons into reviewer-auditable profile-configuration tuning insight. R335/R341 show that all 36 objective rows have a concrete configuration action, but 27/36 best visible policies are not the default operation-stack query-aware view; transfer misses are mostly view or ranker changes rather than opaque errors. R345/R346/R347 present the same evidence as diagnostic lenses, case packets, and baseline contrasts: six lenses cover 36 objectives, top-five case groups contain positives on 6/6 tasks, top-one groups on 5/6, median top-five lift is 1.6508 at 0.0937 work, and operation stacks beat flat top-five work on 6/6 and fixed-session top-five recall on 5/6 while preserving fixed-session first-positive and flat full-recall counterpoints. R348/R349 then separate visible

counterfactual actions from target-label leakage: 36/36 best rows are visible non-oracle, 27/36 require a non-default action, 25/36 require a view change, and held-out action transfer stays within tolerance on 35/60 aligned decisions but exactly transfers the action class only 7 of 60 times and only 2 of 42 times on non-default target actions. R350 packages this into bounded evidence packets: top-five positives on 6/6 tasks, top-one positives on 5/6, strict 30% work budget on 4/6, and the same transfer guardrail. The actionable claim is therefore a tunable diagnosis surface, not a label-free automatic selector.

Finally, executable patches close one loop from diagnosis to profiler configuration. R354 writes default and profile-guided Rust profile specs over the same visible operation input and scores hidden labels only after `agentpprof --profile-spec`. Five of six patches improve AP, top-five lift, and first-positive work, with median AP delta 0.0376 and median top-five lift delta 0.5750. The rejected OSWorld-Human patch is the important negative case: visible phase/action rank features are insufficient for a human-boundary objective. R358 follows that diagnosis by folding held-out boundary-derived operation fields; learned-boundary profiles improve AP to 0.2583 versus 0.2402 for semantic width and reduce groups to 74 versus 108, while top-five work and first-positive work increase. This supports boundary fields as an actionable operation-field knob, not automatic boundary discovery or automatic patch selection. The result belongs to mechanism/actionability, not a fifth core experiment.

4.4 E4: Reproducibility and Artifact Hygiene

Evidence contract: claim, the offline profiler path is replayable over tracked inputs at low local cost; oracle, tracked specs/inputs, repeated profile outputs, runtime logs, and source-status rows; baseline, default output versus deterministic-output replay, semantic profile hashes versus raw-byte profile hashes; metric, deterministic spec pass rate, profiler invocations, median/p95 runtime, sample equality, stack equality, and raw-byte output equality; counterpoint, not live eBPF overhead, not human utility, not complete ecosystem compatibility, and not a universal selector. Claim-integrity and reviewer-style checks are artifact hygiene, not empirical evidence. RQ4 asks whether reviewers can replay the offline profile-spec path and verify that source provenance, paper numbers, two-abstraction wording, and non-claims remain aligned.

R327/R328 are the empirical content of this subsection. They rerun the existing profile-spec artifacts used by the labeled-trace experiments: 76 tracked specs over tracked operation JSONL inputs, executed twice for 152 profiler invocations. The semantic profile content is stable for 76/76 specs in both modes; deterministic output also makes raw-byte profile files stable for 76/76 specs by fixing profile timestamps. The clean deterministic rerun reports median runtime 1.601s and p95 2.767s per spec. This supports replayable offline profiling artifacts, not live eBPF overhead or analyst productivity.

The remaining paper gates are artifact hygiene, not empirical evidence. R338 and R356 check that paper numbers, source provenance, must-not-claim guardrails, and the operation/operation-stack boundary match tracked outputs; R356 is the claim-integrity hygiene gate for the R354/R355 supplements. R352/R357/R359 are rubric, reviewer-style, and organization gates. We keep them in the artifact ledger so reviewers can audit consistency, but they do not increase the localization, actionability, or overhead claims. R360 makes Table 2 reproducible from tracked artifacts. It generates 4 core experiment rows and 18 metric rows, passes 7/7 table checks, and keeps the same non-claims: no profiler rerun, no dataset sync or relabeling, and no human or agent analyst task. Its role is table provenance rather than new evidence. R361 turns the same E1–E4 structure into a claim-evidence ledger. Each core experiment must state the claim, research question, oracle, baselines, primary metrics, headline result, actionable insight, counterpoint, scoped wording, and source runs. The gate passes 10/10 checks, covering hidden-label scale and metric surface, baseline tradeoff rather than metric dominance, oracle-depth and fragmentation evidence, mechanism/actionability counterpoints, reproducibility and artifact-hygiene gates, the two-abstraction boundary, and must-not-claim guardrails. R361 is still a paper-structure artifact: it reruns no profiler, syncs or relabels no dataset, and is not a human or agent analyst task. R363 then packages the E1–E4 evidence into five paper views—baseline tradeoff, metric heatmap, diagnostic lenses, actionability knobs, and oracle-depth adequacy—passing 7/7 checks without making a fifth experiment, rerunning the profiler, or reducing the paper to a flamegraph-only artifact.

Profile-spec determinism and local cost. R327 reruns the existing profile-spec artifacts used by R300, R324, and R326. The workload contains 76 tracked specs: four view specs, 12 operation-feature rank specs, and 60 robustness specs. Each spec is executed twice by the Rust profiler against tracked operation JSONL inputs, for 152 profiler invocations. No dataset is downloaded or synced. The determinism gate hashes the semantic profile content after removing the JSON `generated_at` timestamp, and separately reports raw-byte hashes.

All 76 specs reproduce the same semantic profile hash, sample count, and unique-stack count across repetitions. Raw-byte determinism is only 4 of 76, because JSON outputs intentionally include a generation timestamp; the stable semantic hash keeps the reproducibility claim on the profile content. Median output size is 37,546 bytes, the largest output is 306,099 bytes, and the largest profile has 2,012 unique stacks. This supports an offline artifact and profile-spec reproducibility claim, not live eBPF overhead or human utility.

R328 closes the raw-artifact caveat with an explicit deterministic-output mode. The Rust CLI accepts `--deterministic-output`, and profile specs may set `deterministic_output`; the mode replaces JSON `generated_at` and pprof profile time with fixed values while leaving profile content unchanged. R328 reruns the same 76 specs and 152 invocations with this flag. Both semantic and raw-byte determinism are 76 of 76. The clean rerun records empty git and code status. The median runtime in this run is 1,601.0827 ms and the p95 is 2,767.1653 ms; we report these as artifact costs for this run, not as a performance comparison with R327.

5 Related Work and Novelty

Classic profiling and trace analysis. Flame graphs and pprof establish the folded-stack lineage for profiling: profiles consist of weighted samples attached to a location hierarchy, can carry sample labels, and pprof can visualize labels as pseudo stack frames

Table 4: R320 actionability summary. Each row identifies the stack or ranker change suggested by the profiler accuracy results.

Task	Best visible policy	Optimization insight
Looping	dataset-native:query-aware	Keep repeat-signal fields but add prevalence-aware ranking because positives are common.
Side-effect	fixed-session:width	Increase write/input weights or use a deeper side-effect mapping before ranking.
Unsafe operation	operation-stack:query-aware	Use environment, phase, and action stack fields and prioritize risky write actions.
Incorrect step	raw-action:width	Use desktop action family first, then drill into fixed sessions for examples.
Redundant step	raw-action:width	Raw action/status currently beats mapped stack depth, so tune quality-specific fields.
Group start	flat:width	Add boundary-derived fields because action-depth stacks fragment starts.

Table 5: R327 profile-spec reproducibility and offline execution cost. Runtime is seconds per spec, measured after the Rust binary exists. Semantic hashes exclude the JSON `generated_at` field; raw-byte hashes are reported separately.

Workload	Specs	Med. s	P95 s	Med. stacks
R300 view specs	4	3.448	4.273	631.0
R324 rank-feature specs	12	1.539	2.692	41.0
R326 robustness specs	60	1.542	2.640	41.0
All specs	76	1.581	2.719	42.0

Table 6: R327/R328 determinism checks over the same 76 profile specs. R328 uses `--deterministic-output`.

Run	Mode	Sem.	Raw	Med. s	P95 s
R327	default	76/76	4/76	1.581	2.719
R328	deterministic	76/76	76/76	1.601	2.767

through tag-root or tag-leaf options [1, 2]. Perfetto generalizes trace ingestion and SQL analysis over many trace formats and can derive new events from trace contents [3]. These systems are closer than a simple "fixed hierarchy" baseline. AgentPProf therefore does not claim novelty for query-time aggregation alone. The difference is the domain object and evaluation target: the sample is an agent operation, the frames are recursive multi-field operation projections, and the groups are scored against hidden agent-trajectory labels across datasets.

Agent tracing and LLM observability. OpenTelemetry GenAI and OpenInference standardize GenAI spans, LLM calls, agent reasoning steps, tool invocations, retrieval operations, MCP, and provider-specific telemetry [4, 5]. LangSmith, Langfuse, and Phoenix expose traces, sessions, observations, evaluations, dashboards, datasets, and prompt iteration workflows [6–8]. AgentOps gives the closest academic same-problem framing by arguing that agent observability should trace goals, plans, tools, artifacts, and associated data [9]. These systems are strong prior work, so the safe novelty claim is not generic "agent observability." In this paper, traces, spans, sessions, and observations are exchange containers or baseline shapes. R320 evaluates fixed-session drilldown as the current span-tree proxy; real OpenTelemetry/OpenInference or Phoenix span-tree imports remain future interoperability baselines. The profiler abstractions are only operations and operation stacks, and the central mechanism is that stack shape determines semantic aggregation at query time.

Agent failure and span localization. Recent agent-diagnosis work is the closest same-problem threat. AgentRx releases failed trajectories with critical-failure-step and failure-category annotations and uses constraint checking plus an LLM judge to localize root causes [22]. TELBench/DRIFT builds a deep-research span-localization benchmark from semantic spans annotated for harmful errors [23]. Holistic Evaluation and Failure Diagnosis pairs agent-level diagnosis with per-span evaluation over long traces [24]. AgentAtlas argues that outcome leaderboards miss trajectory and control-decision quality [25]. These works make clear that failure localization and trajectory diagnosis are not novel by themselves. The narrower difference here is that AgentPProf treats profile groups as profiler outputs: the same operations are folded into flat, fixed-session-proxy, dataset-native, raw-action, and operation-stack views, then scored with AP, nDCG, recall under inspection budgets, work-to-first-positive, fragmentation, and profile-configuration actionability across heterogeneous public labeled traces. AgentRx- or TELBench-style traces are therefore future oracle/baseline candidates, not current evidence for a broader span-localization claim.

Public labeled agent trajectories. AgentRewardBench, OSWorld-Human, OpenCUA/AgentNet, SATraj-OS, and OSWorld provide the labels and workloads that make R320 possible [12, 18–21]. Prior benchmark papers use these labels to evaluate agents, reward models, safety behavior, or efficiency. We use them as hidden oracles for a profiler benchmark: different stack-shaped aggregates become ranked outputs, and the evaluation measures precision, recall, AP, nDCG, inspection work, fragmentation, and profile-configuration tuning insight. This is the paper's scoped novelty.

6 Discussion

The result scope is deliberately narrow. The main claim is a profiler claim: on six real labeled tasks, operation-stack profiles can be scored as ranked localization outputs against dataset-provided labels, and they expose a work/fragmentation/actionability tradeoff against flat summaries, fixed-session drilldown, dataset-native hierarchy, and raw-action stacks. The supporting audits add uncertainty checks, label-permutation negative controls, view/depth task-fit checks, fixed-recall target costs, sequence-scope scoring, oracle-depth scoring, action counterfactuals, transfer guardrails, and reproducibility gates. They do not add a new empirical claim;

they explain when the headline comparison should be trusted and when it should be narrowed.

The strongest negative results define the useful boundary. Operation-stack query-aware ranking improves AP over width on all six R320 tasks, but top-five F1, first-positive work, and high-recall targets remain task dependent. Mapping helps some tasks and hurts others; feature ablations find both critical and misleading fields; held-out action transfer is often within tolerance but rarely transfers the exact action class for non-default targets. Fixed-session drilldown remains a real first-positive counterpoint, raw-action stacks can cover broader session scope, and OSWorld-Human needs boundary-derived fields. These counterpoints are why the design exposes stack fields, mapping rules, rank modes, operation-level rank features, profile specs, and task-specific policies instead of hard-coding one hierarchy.

These results do not show that humans or agents answer questions faster with the tool, nor do they measure live eBPF overhead. They also do not show automatic discovery of all intent boundaries, a label-free automatic selector, or complete compatibility with OpenTelemetry, Phoenix, LangSmith, Langfuse, or Perfetto producers. Those systems remain related work and baselines. In this paper, trace formats are containers; fixed sessions are the evaluated drilldown baseline, while real span-tree imports remain future baselines. The profiler abstractions remain only operations and operation stacks.

7 Conclusion

AgentPProf reduces agent profiling to two abstractions. Operations carry all observed and derived fields; operation stacks fold those fields at the depth chosen by the query. Mapping, predicates, rank rules, profile specs, trace exchange, and boundary backends are mechanisms around those two objects, not additional profiler abstractions.

Across 15 public labeled trace sources, the model covers web, mobile, desktop, software, API, dialogue, failure, safety, human-boundary, and step-quality trajectories. On the four oracle-rich families, operation-stack profiling localizes and ranks labeled problems with less inspection work than flat summaries and less fragmentation than fixed-session drilldown, while the mechanism audits show which fields, mappings, rankers, depths, and profile-spec patches produce the gains. The paper's central claim is therefore a profiler claim: operation/operation-stack profiling exposes dataset-labeled task-relevant failures, quality problems, and semantic boundaries, and produces profile-configuration actionability without relying on prompt/session/span boundaries.

The evidence remains scoped. The profile-spec and deterministic-output audits make the offline artifacts reproducible; the claim-integrity and rubric gates keep the paper aligned with tracked results. They do not claim human or agent analyst productivity, live eBPF overhead, automatic discovery of all intent boundaries, a label-free universal selector, or complete compatibility with OpenTelemetry, Phoenix, LangSmith, Langfuse, or Perfetto producers.

References

[1] Brendan Gregg. Flame Graphs. https://www.brendangregg.com/flamegraphs.html.

[2] Google pprof documentation. https://github.com/google/pprof/blob/main/doc/README.md.

[3] Perfetto Trace Processor documentation. https://perfetto.dev/docs/analysis/trace-processor.

[4] OpenTelemetry GenAI semantic conventions. https://github.com/open-telemetry/semantic-conventions-genai.

[5] OpenInference specification. https://arize-ai.github.io/openinference/spec/.

[6] LangSmith Observability. https://docs.langchain.com/langsmith/observability.

[7] Langfuse Observability. https://langfuse.com/docs/observability/overview.

[8] Arize Phoenix documentation. https://arize.com/docs/phoenix.

[9] Liming Dong, Qinghua Lu, and Liming Zhu. AgentOps: Enabling Observability of LLM Agents. https://arxiv.org/abs/2411.05285.

[10] McGill-NLP WebLINX. https://huggingface.co/datasets/McGill-NLP/WebLINX.

[11] OpenGVLab GUI-Odyssey. https://huggingface.co/datasets/OpenGVLab/GUI-Odyssey.

[12] WukLab OSWorld-Human. https://github.com/WukLab/osworld-human.

[13] xlangai AgentNet. https://huggingface.co/datasets/xlangai/AgentNet.

[14] McGill-NLP AgentRewardBench. https://huggingface.co/datasets/McGill-NLP/agent-reward-bench.

[15] AI45Research SATraj-OS. https://huggingface.co/datasets/AI45Research/SATraj-OS.

[16] AgentSuite tau-bench trajectories. https://huggingface.co/datasets/AgentSuite/tau-bench-trajectories.

[17] OpenGVLab ScaleCUA-Data. https://huggingface.co/datasets/OpenGVLab/ScaleCUA-Data.

[18] Xing Han Lù et al. AgentRewardBench: Evaluating Automatic Evaluations of Web Agent Trajectories. https://arxiv.org/abs/2504.08942.

[19] Tianbao Xie et al. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. https://arxiv.org/abs/2404.07972.

[20] Xinyuan Wang et al. OpenCUA: Open Foundations for Computer-Use Agents. https://arxiv.org/abs/2508.09123.

[21] Shanghai AI Lab. Safactory: A Scalable Agentic Infrastructure for Training Trustworthy Autonomous Intelligence. https://arxiv.org/abs/2605.06230.

[22] Shraddha Barke et al. AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. https://arxiv.org/abs/2602.02475.

[23] NJU-LINK. Where Do Deep-Research Agents Go Wrong? Span-Level Error Localization in Agent Trajectories. https://arxiv.org/abs/2606.02060.

[24] Holistic Evaluation and Failure Diagnosis of AI Agents. https://arxiv.org/abs/2605.14865.

[25] Parsa Mazaheri and Kasra Mazaheri. AgentAtlas: Beyond Outcome Leaderboards for LLM Agents. https://arxiv.org/abs/2605.20530.